ACTIVIDAD PRACTICA

# Diseño arquitectural del sistema con IA

## Guía de trabajo para alumnos

En esta actividad convertirán el análisis de negocio elaborado en la Unidad 2 en una propuesta de arquitectura de software. Trabajarán con la misma empresa y el mismo caso. La inteligencia artificial podrá apoyar el diseño y la revisión, pero cada decisión deberá justificarse con evidencia del proyecto.

| | |
|---|---|
| **Empresa asignada** | EVENTOS NOVA |
| **Integrantes** | José Uriel Hernández Hernández 2226704<br>Carlos Tomas Cárdenas Vázquez 2226997<br>Pamela López Martínez 2226900<br>Jaime Guerra Jr 2227428 |
| **Fecha de entrega** | 15 DE OCTUBRE DEL 2026 |



Cisco Confidential

# 1 Proposito de la actividad

El objetivo es organizar las reglas y responsabilidades del sistema antes de programarlo por completo. Al terminar, el equipo deberá demostrar cómo pasó de los requerimientos de la Unidad 2 a una arquitectura que separa el dominio, la aplicación y la infraestructura.

La actividad no consiste en pedir a una herramienta de IA que produzca una solución terminada. El trabajo consiste en plantear decisiones, solicitar alternativas, revisar las respuestas y documentar por qué una propuesta sí corresponde o no al negocio analizado.

## Resultados de aprendizaje

- Comparar Transaction Script, Table Module y Domain Model con base en la complejidad real del caso.
- Modelar el dominio mediante lenguaje ubicuo, entidades, Value Objects, Aggregates y Domain Events.
- Diseñar una Service Layer que coordine casos de uso sin absorber las reglas del dominio.
- Separar la persistencia mediante Repository y definir el alcance transaccional con Unit of Work.
- Distinguir Commands y Queries en una propuesta básica de CQRS.
- Evaluar la arquitectura con los principios SOLID y registrar las decisiones en ADR.
- Usar IA como copiloto y conservar evidencia de la revisión humana.

# 2 continuidad con la Unidad 2

Trabajarán con la misma empresa y no comenzarán un caso nuevo. Antes de diseñar la arquitectura, reúnan los productos elaborados en la Unidad 2 y asegúrense de que todos los integrantes comprenden el proceso principal del negocio.

| Insumo de la Unidad 2 | Como se utilizará en la Unidad 3 |
|---|---|
| BRD | Delimitar alcance, actores, necesidades, restricciones, supuestos y criterios de éxito. |
| Historias de usuario y Gherkin | Identificar casos de uso, condiciones de negocio y comportamientos esperados. |
| BPMN | Reconocer la secuencia, decisiones, participantes, excepciones y puntos de coordinación. |
| Glosario y modelo de dominio | Mantener un lenguaje ubicuo y localizar conceptos con identidad, valor o ciclo de vida. |
| Matriz de trazabilidad | Relacionar cada decisión arquitectural con una necesidad, historia, regla o escenario. |
| Repositorio de GitHub | Conservar versiones, diagramas, documentos, prompts y código generado o revisado. |

## Regla de continuidad

Si el equipo detecta una contradicción o un dato faltante en la Unidad 2, deberá registrarlo como hallazgo. Puede ajustar el análisis anterior, pero debe explicar el cambio y actualizar la trazabilidad. No se permite inventar nuevas reglas solo para acomodar una arquitectura.



Cisco Confidential

# 3 Empresas de trabajo

Cada equipo continuará con la empresa que trabajó en la Unidad 2. La siguiente tabla recuerda el contexto general de los nueve casos. Las preguntas sirven para enfocar la revisión; no indican qué patrón, clases o componentes deben elegir.

| Empresa | Contexto | Pregunta de analisis |
|---|---|---|
| Mercado Norte | Alimentos, perecederos y promociones | ¿Que reglas cambian por producto, sucursal, fecha o existencia? |
| TecnoNova Retail | Electronica, numeros de serie y garantias | ¿Que decisiones requieren conservar identidad y ciclo de vida? |
| Moda Urbana | Ropa, tallas, colores y temporadas | ¿Donde aparecen variantes, reservas, cambios y devoluciones? |
| HogarMax | Hogar, unidades voluminosas y ventas a credito | ¿Que operaciones coordinan inventario, credito y entrega? |
| SaludPlus Farmacias | Medicamentos, lotes y caducidad | ¿Que invariantes deben cumplirse antes de autorizar una operacion? |
| SportZone | Articulos deportivos y comisiones | ¿Que reglas pertenecen a la venta y cuales a procesos posteriores? |
| Pet and Co | Productos, servicios y suscripciones | ¿Que comportamientos dependen del plan, vigencia o tipo de servicio? |
| Casa and Estilo | Muebles, anticipos y entregas | ¿Que estados y eventos cambian durante el cumplimiento de un pedido? |
| LibroMundo | Libros, ISBN y preventas | ¿Que cambia entre una venta normal, una preventa y una cancelacion? |

# 4 Reglas generales

- Mantengan el alcance aprobado en el BRD y seleccionen un proceso principal para el diseño detallado.
- Todas las decisiones deben relacionarse con requerimientos, reglas, historias o escenarios existentes.
- No se evaluará la cantidad de clases. Se evaluará si cada responsabilidad está ubicada y justificada correctamente.
- Los nombres usados en diagramas, código y documentos deben coincidir con el glosario del negocio.
- El código puede ser parcial o boilerplate. Debe permitir revisar la organización y las responsabilidades propuestas.
- Conserven las iteraciones realizadas con IA. No entreguen una respuesta generada sin revisión.



Cisco Confidential

# 5 Ruta de trabajo

Realicen las etapas en orden. Una etapa puede obligarlos a corregir una decisión anterior; registren el cambio en lugar de ocultarlo.

| Etapa | Trabajo | Resultado |
|---|---|---|
| 1 | Preparar el caso | Revisar U2, delimitar proceso y seleccionar casos de uso. |
| 2 | Elegir el patrón | Comparar alternativas y justificar la opción elegida. |
| 3 | Construir el mapa de dominio | Definir lenguaje, responsabilidades, limites y eventos. |
| 4 | Diseñar la Service Layer | Organizar la coordinacion de los casos de uso. |
| 5 | Separar persistencia y transacciones | Proponer Repository y Unit of Work. |
| 6 | Separar comandos y consultas | Aplicar CQRS basico donde tenga sentido. |
| 7 | Revisar con SOLID | Detectar acoplamiento y responsabilidades incorrectas. |
| 8 | Representar la arquitectura | Crear UML, secuencias y ADR. |
| 9 | Validar y entregar | Trazar, probar, depurar y organizar evidencias. |

# 6 Etapa 1 Preparar el caso

Seleccionen de tres a cinco casos de uso relacionados con el proceso principal. Deben incluir al menos un flujo normal y un flujo con excepción o rechazo. Para cada caso de uso, identifiquen el actor, el objetivo, la información de entrada, las reglas involucradas y el resultado esperado.

| ID | Caso de uso | Actor | Reglas relacionadas | Resultado esperado |
|---|---|---|---|---|
| CU-01 | Aprobación de cotización (US-04) | Coordinador | Generar folio, estado "Pendiente", calcular 25% de anticipo. | Propuesta cambia a "Enviada" y se calcula el anticipo automático. |
| CU-02 | Cancelación y lista de advertencia (US-07) | Sistema | Detectar cancelación de proveedor, sugerir reemplazo, clasificar. | Alerta enviada al Coordinador y proveedor etiquetado en lista negra |



Cisco Confidential

| ID | Caso de uso | Actor | Reglas relacionadas | Resultado esperado |
|---|---|---|---|---|
| CU-03 | Limite de cambios normales (US-08) | Coordinador | Límite estricto de 10 días previos al evento para modificaciones. | Bloqueo automático de modificación; exige autorización especial |
| CU-04 | Bloqueo de presupuesto (US-11) | Sistema | Bloquear servicios adicionales que excedan presupuesto inicial. | Botón de guardar desactivado y muestra error por límite financiero. |
| CU-05 | Penalización por saldo pendiente (US-13 | Administrador | Tolerancia de 2 días post-fecha de liquidación, cancelación en cascada. | Servicios marcados para cancelación, aplicación de cobro extra. |

Antes de continuar, comprueben que los casos seleccionados aparecen en las historias de usuario o en el BPMN. Si no existe esa relación, el caso debe justificarse como corrección de alcance.

# 7 Etapa 2 Elegir el patrón de lógica de negocio

Comparen Transaction Script, Table Module y Domain Model. La decisión deberá basarse en el comportamiento del sistema: cantidad de reglas, interacción entre conceptos, cambios esperados, manejo de estados y necesidad de proteger invariantes.

## Trabajo solicitado

- Construyan una tabla comparativa con criterios propios del caso.
- Incluyan ventajas, riesgos y consecuencias de cada alternativa en su empresa.
- Seleccionen un patrón principal. Pueden proponer una combinación solo si delimitan dónde se usa cada patrón.
- Relacionen la decisión con al menos tres evidencias de la Unidad 2.

| Criterio del caso | Transaction Script | Table Module | Domain Model | Evidencia U2 |
|---|---|---|---|---|
| Complejidad de reglas | Pobre. Mezcla validaciones de negocio con sentencias de base de datos. | Medio. Útil para CRUDs, difícil al aplicar reglas entre múltiples entidades cruzadas. | **Excelente.** Aísla reglas complejas (límites de tiempo, penalizaciones) en objetos propios | BRD (Restricciones) y US 08, 11, 13. |
| Cambios esperados | Difícil. Alterar reglas de anticipo obliga a buscar y editar múltiples scripts. | Moderado. Estructura altamente acoplada al esquema de la base de datos. | **Alto.** Modificar lógicas de cancelación o presupuesto no afecta la capa de persistencia | BRD (Necesidades del Negocio). |



Cisco Confidential

| Criterio del caso | Transaction Script | Table Module | Domain Model | Evidencia U2 |
|---|---|---|---|---|
| Interacción entre conceptos | Riesgo de código espagueti al cruzar Cliente, Contrato y Proveedor en un solo script. | Requiere manejar la lógica mediante múltiples RecordSets asociados. | **Fluida.** Los objetos (Aggregates) controlan y validan sus propias dependencias internas | Modelo de Dominio y BPMN. |
| Estados e invariantes | Nulo. No protege transiciones de estado de forma nativa. | Delega la protección de invariantes a restricciones puramente de la BD (SQL). | **Ideal.** Protege invariantes críticos como "Contrato Activo" o "Expirado" directamente en la clase | Glosario (Definición de Contrato Activo). |
| Criterio adicional | Los cálculos de fechas y penalizaciones se programan directamente en funciones sueltas, lo que genera duplicidad y riesgo de errores si las reglas cambian. | La validación se hace a nivel de tabla, pero se complica cuando el límite de tiempo depende del estado de otro registro (ej. fecha de Contrato vs fecha de Evento). | **Excelente.** Los objetos manejan su propio ciclo de vida. Una entidad Contrato calcula internamente si aún está en el periodo de gracia o si ya aplica penalización. | BRD (Restricciones temporales y financieras) y Glosario (Límite de Cambios Normales) |

Producto de la etapa: una decisión argumentada de una a dos páginas. No basta escribir que un patrón es más fácil, moderno o escalable. La explicación debe indicar qué problema concreto del caso ayuda a resolver y qué costo introduce.

**Decisión argumentada:**

Tras analizar la complejidad operativa de Eventos Nova y evaluar los patrones Transaction Script, Table Module y Domain Model, en la tabla comparativa anterior. El equipo ha decidido implementar **Domain Model** como la arquitectura central de lógica de negocio. Esta decisión no se basa únicamente en la escalabilidad o modernidad del patrón, sino en la necesidad estricta de manejar los múltiples estados, transiciones y reglas financieras que se establecieron anteriormente para el negocio.

La justificación por la cual se eligió ese modelo radica en los siguientes problemas específicos extraídos del BRD de la unidad 2.

**Protección de Invariantes Financieros y Presupuestales:** Eventos Nova sufre actualmente por una fuerte dificultad para controlar los costos y los cambios autorizados. Una regla de negocio estricta (US 11) dicta que el sistema debe bloquear automáticamente los servicios adicionales si estos superan el presupuesto inicial aprobado. Usar un Transaction Script dejaría esta validación expuesta en la capa de interfaz o en funciones procedimentales dispersas. Con Domain Model, el agregado Contrato o Evento se convierte en el único responsable de autorizar adiciones. El modelo protege el invariante impidiendo que la lista de servicios crezca si el CostoTotal excede el PresupuestoAprobado, lanzando una excepción de dominio antes de siquiera tocar la base de datos.



Cisco Confidential

**Gestión de Reglas Temporales Complejas:** La operación de Nova depende de límites de tiempo: los proveedores tienen de 2 a 3 días para confirmar; los clientes tienen hasta 10 días previos al evento para solicitar cambios de menú o cancelaciones; y existen penalizaciones por saldos pendientes tras 2 días de periodo de gracia. Un Table Module o Transaction Script obligaría a crear scripts de bases de datos o rutinas externas muy pesadas para calcular estas fechas en cada consulta. El Domain Model resuelve esto encapsulando la lógica temporal. El objeto Evento conoce su propia fecha de ejecución y posee métodos como SolicitarCambio(), el cual internamente valida si la diferencia entre la fecha actual y la fecha del evento es mayor a 10 días. Si no lo es, el objeto mismo rechaza la modificación y protege la integridad del estado.

**Transiciones de Estado de Proveedores (Lista de Advertencia):** Cuando un proveedor cancela de forma repentina, no solo se debe actualizar la base de datos, sino que el sistema debe desencadenar sugerencias de reemplazo y mover al proveedor a una "lista de advertencia" (US 07). El Domain Model facilita la emisión de Domain Events (Eventos de Dominio). Cuando la entidad Proveedor ejecuta el método CancelarParticipacion(), dispara un evento que la capa de aplicación escucha para notificar al Coordinador, logrando una arquitectura reactiva y sin acoplamiento duro.

**Costos técnicos y desventajas introducidas**

La elección de Domain Model no está exenta de sacrificios. Al elegir este patrón, el equipo acepta las siguientes consecuencias y costos para el proyecto:

- **Curva de aprendizaje:** A diferencia de un simple CRUD (Create, Read, Update, Delete) manejado por Transaction Script, el Domain Model exige crear una gran cantidad de código inicial (clases, interfaces, repositorios, value objects y mapeadores) antes de poder ver una funcionalidad operativa.
- **Complejidad en el Mapeo de Datos:** Dado que la estructura de los objetos de negocio no coincidirá exactamente con las tablas de la base de datos relacional, se introduce la necesidad imperativa de utilizar un ORM (Object-Relational Mapper) avanzado o mapeadores manuales (Patrón Repository) para traducir los objetos de dominio a filas de base de datos. Esto añade sobrecarga de procesamiento.
- **Problemas de Rendimiento en Lecturas Masivas:** Cargar un Aggregate completo (ej. un Evento con sus 20 proveedores y su contrato) a la memoria solo para leer el nombre del cliente resulta ineficiente y costoso en términos de RAM y tiempo de procesamiento.
- **Mitigación:** Para combatir este costo técnico, la arquitectura se apoyará en la **Etapa 6 (Separación de Commands y Queries - CQRS)**, delegando las escrituras y validaciones complejas al Domain Model, pero permitiendo que las consultas de lectura simples ataquen directamente a la base de datos mediante vistas o DTOs, equilibrando así la balanza de rendimiento.

# 8 Etapa 3 Construir el mapa de dominio

Transformen el lenguaje y las reglas de la Unidad 2 en un mapa de responsabilidades. Identifiquen los elementos que el sistema debe reconocer, proteger o comunicar. No conviertan automáticamente cada tabla de datos en una entidad del dominio.



Cisco Confidential

## Elementos que deben identificar

| Elemento | Pregunta que debe responder el equipo |
|---|---|
| Lenguaje ubicuo | Términos clave compartidos por el equipo y el negocio: *Evento, Contrato Principal, Contrato Adicional, Proveedor, Límite de Cambios Normales, Costo Nova, Cargo Extra por Excedente, Lista de Advertencia, Anticipo Inicial*. |
| Entidades | Conceptos con identidad única (ID) que cambian con el tiempo:<br><br>• Evento: Mantiene su identidad, aunque cambie de fecha, cantidad de invitados o estado.<br><br>• Proveedor: Entidad externa registrada en el catálogo con un ciclo de vida propio (Activo, Bloqueado, En Lista de Advertencia).<br><br>• Cliente: Persona o empresa registrada (Nombre, Teléfono, Correo). |
| Value Objects | Conceptos inmutables definidos por su valor (sin ID propio):<br><br>• Presupuesto: Contiene el Monto y la Moneda. Regla: No puede ser negativo.<br><br>• PeriodoConfirmacion: Un lapso (ej. 3 días) para la disponibilidad del proveedor.<br><br>• DatosContacto: Agrupa Teléfono y Correo Electrónico. Si cambia un dígito, es un nuevo contacto. |
| Aggregates | Grupos que garantizan la consistencia (con una raíz):<br><br>**1. Aggregate Root: Contrato**<br>- *Responsabilidad:* Controlar el ciclo de vida financiero y las adiciones al evento.<br>- *Elementos internos:* Contrato Principal, Contrato Adicional, Presupuesto.<br>- *Invariantes:* No se agregan servicios si exceden el Presupuesto (US 11). Solo es "Activo" si se validó el Anticipo (US 05).<br><br>**2. Aggregate Root: Proveedor**<br>- *Responsabilidad:* Administrar disponibilidad, datos de contacto y estatus de confiabilidad.<br>- *Elementos internos:* DatosContacto, PeriodoConfirmacion.<br>- *Invariantes:* Si pasan 3 días sin respuesta, el estado cambia automáticamente a "Expirado" (US 06). |



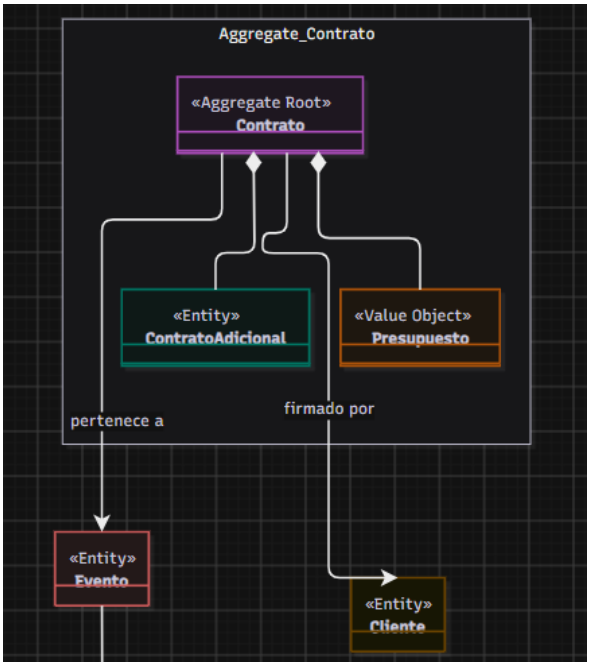
Cisco Confidential

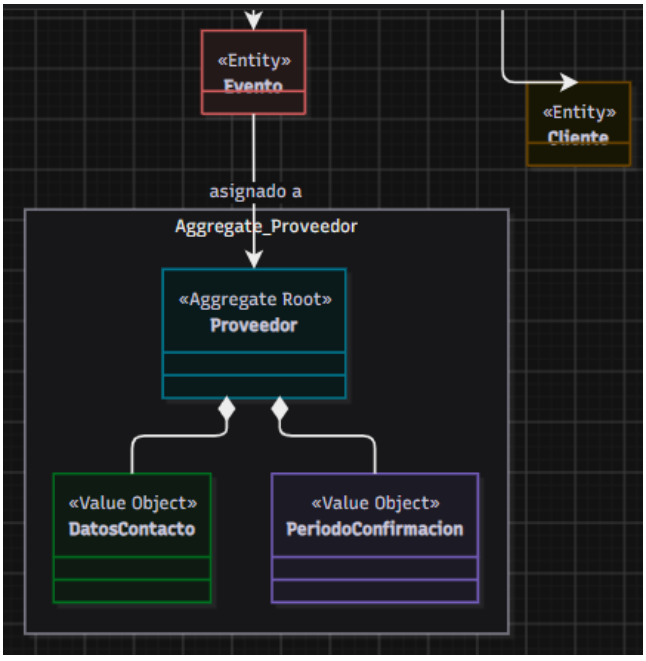
| Elemento | Pregunta que debe responder el equipo |
|---|---|
| Domain Services | Operaciones complejas que no caben en una sola entidad:<br><br>• PenalizacionService: Calcula montos de multa y coordina la cancelación en cascada de múltiples proveedores cuando el cliente no liquida tras el plazo de gracia de 2 días (US 13). |
| Domain Events | Hechos del negocio que ya ocurrieron y pueden interesar a otros componentes.<br>• ContratoActivadoEvent: Se dispara cuando Administración valida el anticipo, notificando a los proveedores.<br><br>• ProveedorCanceladoEvent: Detona la búsqueda de sugerencias de reemplazo y la inserción del proveedor en la Lista de Advertencia (US 07).<br><br>• ServiciosModificadosEvent: Dispara notificaciones automáticas a los proveedores cuando el Coordinador autoriza un cambio (US 09). |

## Producto de la etapa

- Diagrama del mapa de dominio con relaciones y límites claramente identificados.
- Glosario actualizado con los términos nuevos o corregidos.
- Ficha de cada Aggregate que indique su raíz, responsabilidades, invariantes y eventos posibles.
- Explicación de las decisiones dudosas o descartadas.

**DIAGRAMA DEL MAPA DE DOMINIO CON RELACIONES Y LIMITES.**






Cisco Confidential

**Nuevos términos al glosario.**

**Aggregate Root (Raíz de Agregado):** Entidad principal (ej. Contrato) que actúa como el único portón de entrada para modificar los datos internos de su grupo, garantizando que no se rompan las reglas de Eventos Nova.

**Presupuesto Operativo (Value Object):** Representación inmutable del límite de dinero autorizado. No es solo una cifra, contiene la moneda y la lógica para saber si un nuevo servicio supera el límite.

**Lista de Advertencia Activa:** Estado específico dentro del ciclo de vida del Proveedor que se activa mediante un Evento de Dominio tras una cancelación.

**Ficha de cada Aggregate.**

**Ficha Aggregate 1: Contrato Financiero**

- Raíz (Root): Contrato
- Responsabilidades: Controlar el ciclo de vida financiero del evento, validar límites de presupuesto y aprobar la anexión de servicios extra.
- Invariantes (Reglas irrompibles):
    1. El costo total de ContratoAdicional + Contrato jamás puede exceder el Presupuesto autorizado sin autorización explícita.
    2. Un contrato no pasa a estado "Activo" sin registrar el pago exacto del Anticipo Inicial (25%).
- Eventos Posibles: ContratoActivadoEvent, PresupuestoExcedidoEvent.

**Ficha Aggregate 2: Gestión de Proveedores**

- Raíz (Root): Proveedor
- Responsabilidades: Administrar los tiempos de confirmación, historial de fallas y datos de contacto de las entidades externas.
- Invariantes (Reglas irrompibles):
    1. Si no hay respuesta en el PeriodoConfirmacion (3 días), el estado cambia estrictamente a "Expirado".
- Eventos Posibles: ProveedorCanceladoEvent, AdvertenciaGeneradaEvent.

**Explicación de las decisiones dudosas o descartadas**

- **Decisión Dudosa 1:** Se debatió si un ContratoAdicional debía ser su propio Aggregate Root independiente, ya que el cliente puede solicitarlo días después.
- **Por qué se descartó:** Se decidió meterlo como una Entidad "hija" dentro del Aggregate Contrato porque un servicio extra no tiene razón de existir ni ciclo de vida propio si no está amarrado a un evento principal. Además, dejarlo suelto impediría calcular si la suma global excede el presupuesto (US 11).
- **Decisión Dudosa 2:** Convertir el "Contacto" (teléfono/correo) en Entidad.
- **Por qué se descartó:** Se modeló como *Value Object*. Si un cliente cambia su número de teléfono, no editamos el objeto "teléfono", simplemente destruimos ese dato y creamos uno nuevo, porque su identidad real radica en el Cliente, no en su número.



Cisco Confidential

# 9 Etapa 4 Diseñar la Service Layer

Diseñen los Application Services que expondrán los casos de uso. Cada método debe representar una intención completa del usuario o de otro sistema. El servicio coordina la operación: recibe la solicitud, obtiene los objetos necesarios, invoca el comportamiento del dominio, solicita la persistencia y devuelve un resultado.

La Service Layer no debe convertirse en un lugar donde se acumulen todas las reglas. Si una condición describe lo que el negocio permite, calcula o protege, el equipo debe justificar dónde vive esa regla y por qué.

| Servicio | Método o caso de uso | Entrada | Colaboradores | Salida | Reglas que delega |
|---|---|---|---|---|---|
| ContratoAppService | AprobarCotizacion() *(CU-01 / US 04)* | Guid contratoId, Guid coordinadorId | IContratoRepository, IUnitOfWork | CotizacionAprobadaDTO | Delega el cálculo exacto del 25% de anticipo a la entidad Contrato. El servicio no hace matemáticas. |
| ProveedorAppService | RegistrarCancelacion() *(CU-02 / US 07)* | Guid proveedorId, Guid eventoId | IProveedorRepository, IDomainEventDispatcher | CancelacionResultDTO | Delega la lógica de cambiar el estado interno a "Lista de Advertencia" a la entidad Proveedor |
| EventoAppService | SolicitarCambioNormal() *(CU-03 / US 08)* | Guid eventoId, DetalleCambioDTO | IEventoRepository, IUnitOfWork | bool (Éxito) | Delega la validación temporal (revisar si faltan menos de 10 días) a la entidad Evento |
| ContratoAppService | AgregarServicioExtra() *(CU-04 / US 11)* | Guid contratoId, ServicioExtraDTO | IContratoRepository, IUnitOfWork | CostoActualizadoDTO | Delega el bloqueo financiero estricto (validar que Costo Total < Presupuesto) al Aggregate Root Contrato |
| AdminAppService | AplicarPenalizacion() *(CU-05 / US 13)* | Guid contratoId | IContratoRepository, PenalizacionDomainService | PenalizacionResultDTO | Delega la validación de los 2 días de gracia a la entidad Contrato y el cálculo de la multa al Domain Service. |



Cisco Confidential

### Revisión obligatoria

- ¿El nombre del método expresa una acción del negocio?
  Sí. Se descartaron nombres técnicos como UpdateContrato() o PatchProveedor(). En su lugar, se utilizan intenciones claras del lenguaje ubicuo como AprobarCotizacion() y RegistrarCancelacion(), reflejando las acciones de los actores (Coordinador y Administración).
- ¿El método coordina o contiene decisiones que deberían estar en el dominio?
  Solamente coordina. Por ejemplo, en AgregarServicioExtra(), el servicio recupera el contrato de la base de datos y llama al método contrato.AgregarServicio(nuevoServicio). Si se excede el presupuesto, la entidad Contrato lanza una excepción, por lo que el Application Service está libre de sentencias lógicas y matemáticas.
- ¿El servicio depende de interfaces o de detalles concretos de base de datos?
  Depende 100% de abstracciones. Se inyectan colaboradores como IContratoRepository. El servicio no tiene ni idea de si los datos vienen de SQL Server, PostgreSQL o un archivo de texto, protegiendo así la capa de aplicación.
- ¿La operación tiene un inicio y un final transaccional comprensible?
  Sí. El Application Service inicia la transacción, ejecuta la acción del dominio y llama a IUnitOfWork.Commit(). Si el dominio lanza una alerta (ej. límite de 10 días violado), el servicio atrapa el error y la transacción jamás se confirma, asegurando consistencia.
- ¿Los errores del dominio pueden explicarse con términos del negocio?
  Totalmente. En lugar de regresar un error técnico como *"NullReferenceException"* o *"Error 500 DB FK"*, el dominio arrojará excepciones claras de negocio como PresupuestoExcedidoException o CambioFueraDeTiempoException, las cuales se pueden mostrar directamente al Cliente o Coordinador.

# 10 Etapa 5 Separar persistencia y transacciones

Definan interfaces de Repository en términos del dominio y describan, por separado, cómo podrían implementarse en infraestructura. Los métodos deben expresar lo que el dominio necesita recuperar o guardar, sin exponer consultas SQL como parte del modelo.

También indiquen qué operaciones deben completarse juntas. La propuesta de Unit of Work debe explicar dónde inicia la transacción, qué cambios incluye y qué ocurriría ante un error.

| Repository | Necesidad del dominio | Metodos propuestos | Objeto que recupera o guarda |
|---|---|---|---|
| IContratoRepository | Consultar y actualizar contratos, anticipos, presupuestos y penalizaciones. | ObtenerPorId(), Guardar() | Aggregate Contrato, ContratoAdicional y Presupuesto. |
| IProveedorRepository | Administrar proveedores, sus estados y cancelaciones. | ObtenerPorId(), Guardar() | Aggregate Proveedor, DatosContacto y PeriodoConfirmacion. |
| IEventoRepository | Consultar eventos y validar modificaciones según su fecha. | ObtenerPorId(), Guardar() | Entidad Evento, fecha, estado y cambios autorizados. |



Cisco Confidential

# 11 Etapa 6 Separar Commands y Queries

Apliquen CQRS básico para hacer explícita la diferencia entre cambiar el estado y consultar información. No es obligatorio construir bases de datos separadas. El objetivo es que cada operación tenga una responsabilidad clara y que las consultas puedan devolver un modelo apropiado para lectura.

## Trabajo solicitado

- Seleccionen al menos dos operaciones que cambien el estado y descríbanlas como Commands.
- Seleccionen al menos dos necesidades de información y descríbanlas como Queries.
- Definan el Handler de cada operación y sus dependencias.
- Diseñen el DTO o View Model de lectura sin reutilizarlo automáticamente como modelo de escritura.
- Justifiquen si alguna parte del caso no necesita esta separación.

| Tipo | Nombre | Datos de entrada | Handler | Resultado | Justificacion |
|---|---|---|---|---|---|
| Command | AprobarCotizacionCommand | contratoId, coordinadorId | AprobarCotizacionHandler | CotizacionAprobadaDTO | Cambia el estado de la cotización y calcula el anticipo del 25% |
| Command | AgregarServicioExtraCommand | contratoId, ServicioExtraDTO | AgregarServicioExtraHandler | AgregarServicioExtraHandler | Modifica el contrato y valida que no se exceda el presupuesto autorizado |
| Query | ConsultarContratoQuery | contratoId | ConsultarContratoHandler | ConsultarContratoHandler | Permite consultar el estado, presupuesto y anticipo sin modificar información |
| Query | ConsultarProveedoresQuery | eventoId | ConsultarProveedoresHandler | ListaProveedorDTO | Muestra los proveedores relacionados con un evento y sus estados para facilitar el seguimiento |



Cisco Confidential

# 12 Etapa 7 Revisar la propuesta con SOLID

Revisen la propuesta completa antes de cerrar los diagramas. El análisis debe mostrar decisiones concretas y no una definición memorizada de cada principio.

| Principio | Pregunta de revision | Evidencia que deben buscar |
|---|---|---|
| S | ¿Cada clase o servicio tiene una razon principal para cambiar? | Responsabilidades mezcladas, clases extensas, servicios que hacen de todo. |
| O | ¿Pueden agregar una variante de regla sin editar una cadena creciente de condiciones? | Condicionales que deben modificarse por cada nueva politica. |
| L | ¿Las implementaciones pueden sustituirse sin cambiar el comportamiento esperado? | Subtipos que rechazan operaciones o alteran contratos. |
| I | ¿Cada consumidor depende solo de las operaciones que utiliza? | Interfaces grandes con metodos irrelevantes para algunos clientes. |
| D | ¿La aplicacion y el dominio dependen de abstracciones? | Dependencias directas de SQL, frameworks o servicios externos. |

Registren al menos cinco hallazgos. Un hallazgo puede confirmar una buena decisión o señalar una corrección necesaria. Para cada uno, incluyan evidencia, impacto y cambio realizado.

# 13 Etapa 8 Representar y documentar la arquitectura

## Diagrama de clases

El diagrama diferencia las tres capas por color: verde para la Aplicación, azul para Dominio y naranja para Infraestructura. La regla de dependencia es que la aplicación y la infraestructura apuntan hacia abstracciones definidas junto al dominio o la aplicación.

| Capa | Elementos incluidos | Responsabilidad | Etapa |
|---|---|---|---|
| **Dominio** | Aggregate Roots: Contrato, Proveedor. Entidades: Evento, Cliente, ContratoPrincipal, ContratoAdicional. Value Objects: Presupuesto, DatosContacto, PeriodoConfirmacion. Domain Service: PenalizacionDomainService. Domain Events y excepciones de negocio. Interfaces IContratoRepository, IProveedorRepository, IEventoRepository. | Proteger invariantes (presupuesto, límite de 10 días, anticipo de 25 %, tolerancia de 2 días, expiración de 3 días) y emitir hechos del negocio. Contiene las reglas; no conoce la base de datos. | 3 y 5 |
| **Aplicación** | Command Handlers y Query Handlers; Application Services (ContratoAppService, ProveedorAppService, EventoAppService, AdminAppService); interfaces IUnitOfWork, IDomainEventDispatcher, IContratoConsultas, IProveedorConsultas. | Recibir el caso de uso, obtener los Aggregates, invocar el comportamiento del dominio, delimitar la transacción y devolver un DTO o un error de negocio. No calcula ni decide reglas. | 4 y 6 |



Cisco Confidential

| | | | |
|---|---|---|---|
| **Infraestructura** | ContratoRepository, ProveedorRepository, EventoRepository, SqlUnitOfWork, DomainEventDispatcher, ContratoConsultas, ProveedorConsultas. | Implementar las interfaces: traducir Aggregates a filas (ORM o mapeadores), manejar la transacción y resolver las consultas de lectura mediante vistas o DTOs. | 5 y 6 |

## Diagrama de clases (vista general)

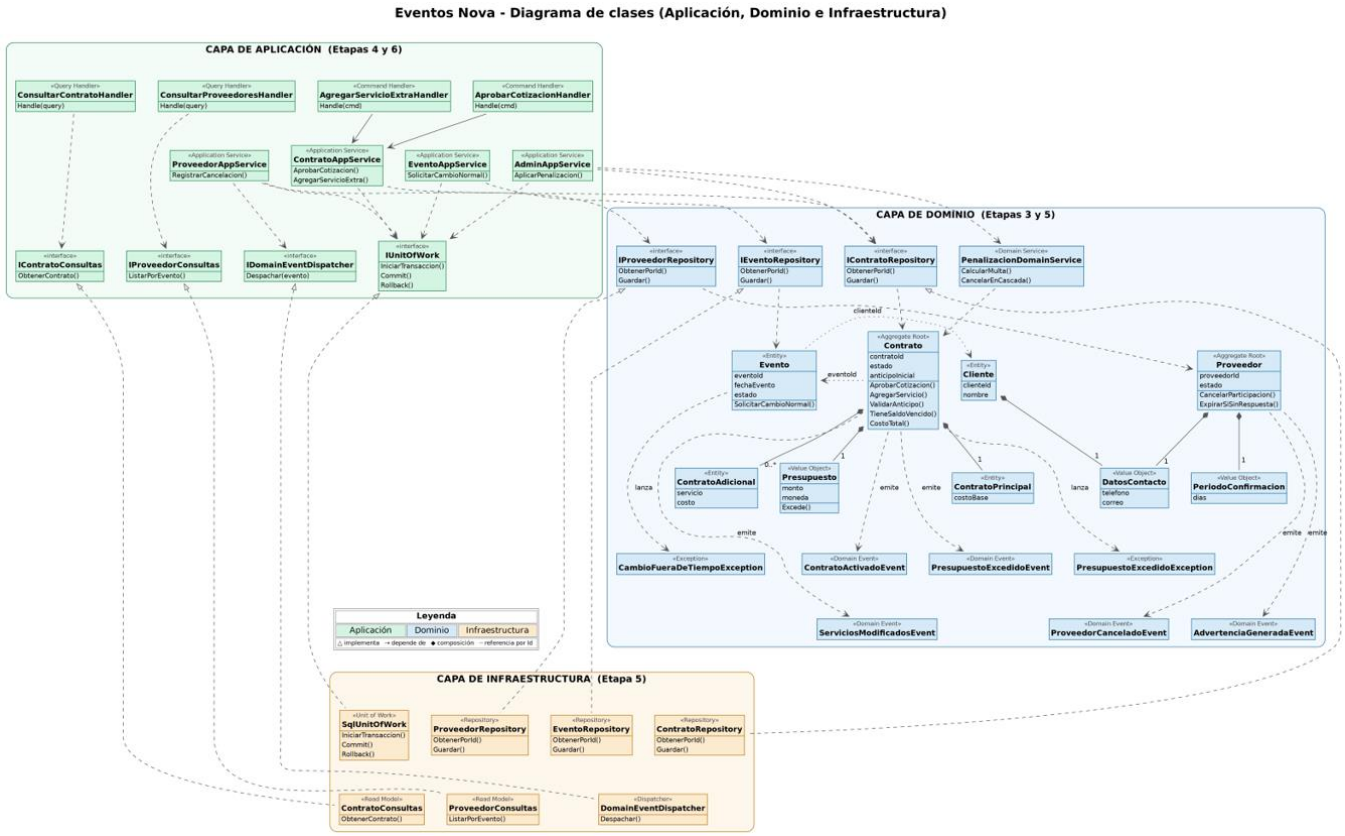



**Figura 1A. Detalle de la capa de Dominio**



Cisco Confidential

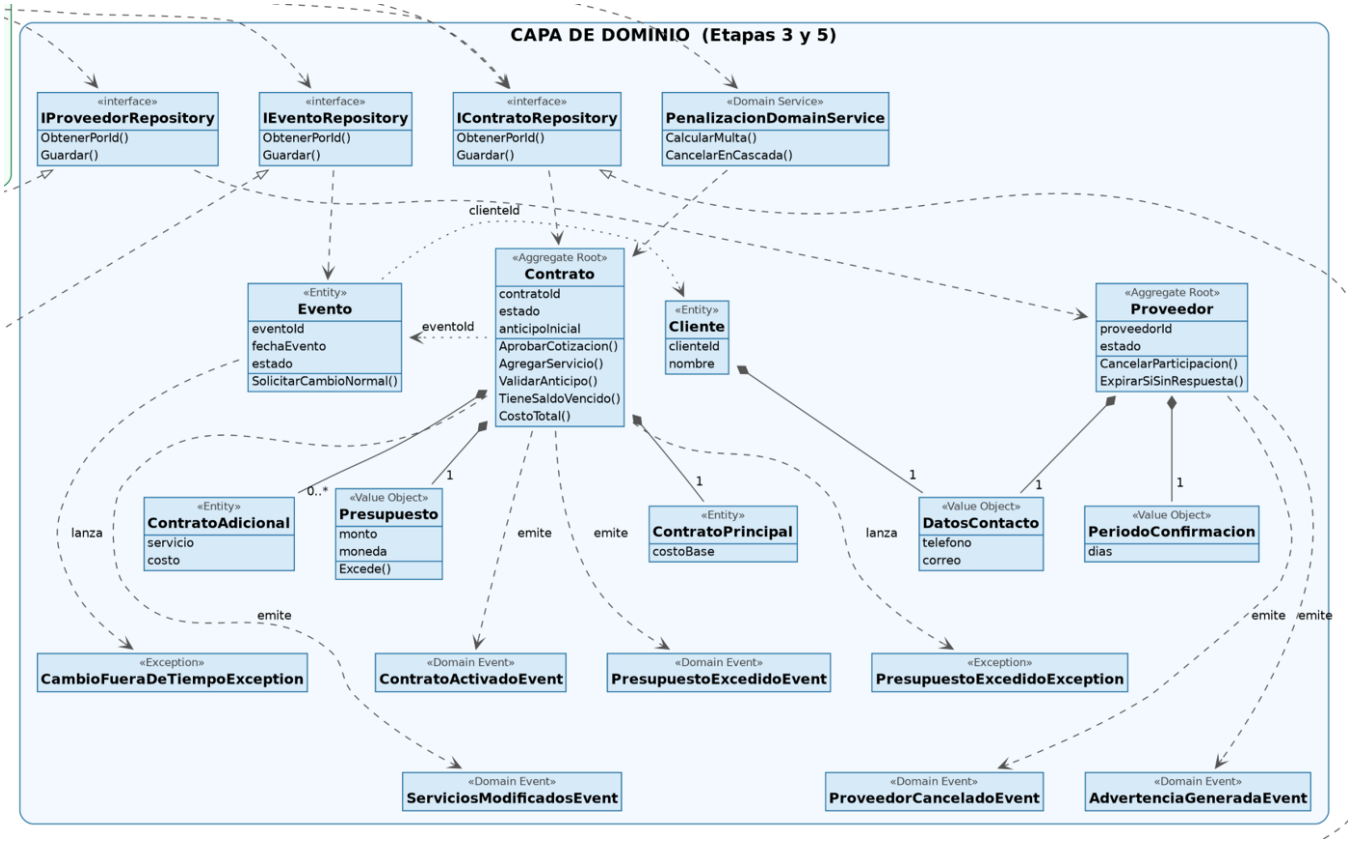



**Figura 1B. Detalle de la capa de Aplicación**

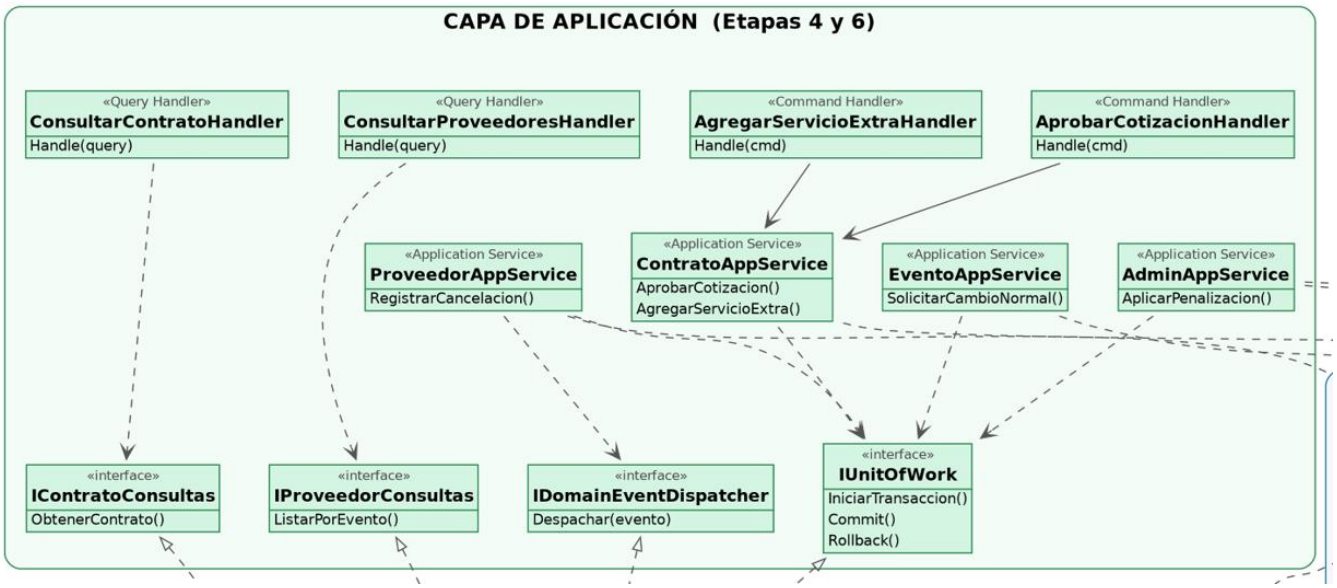
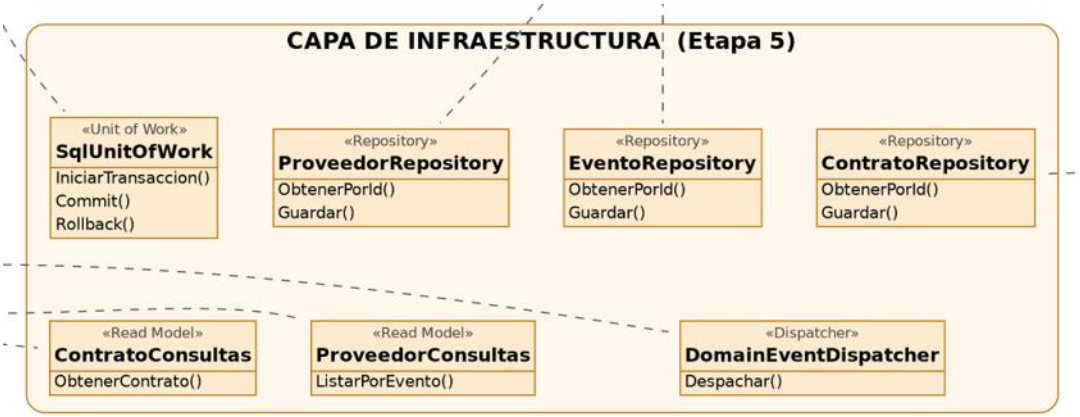



**Figura 1C. Detalle de la capa de Infraestructura**

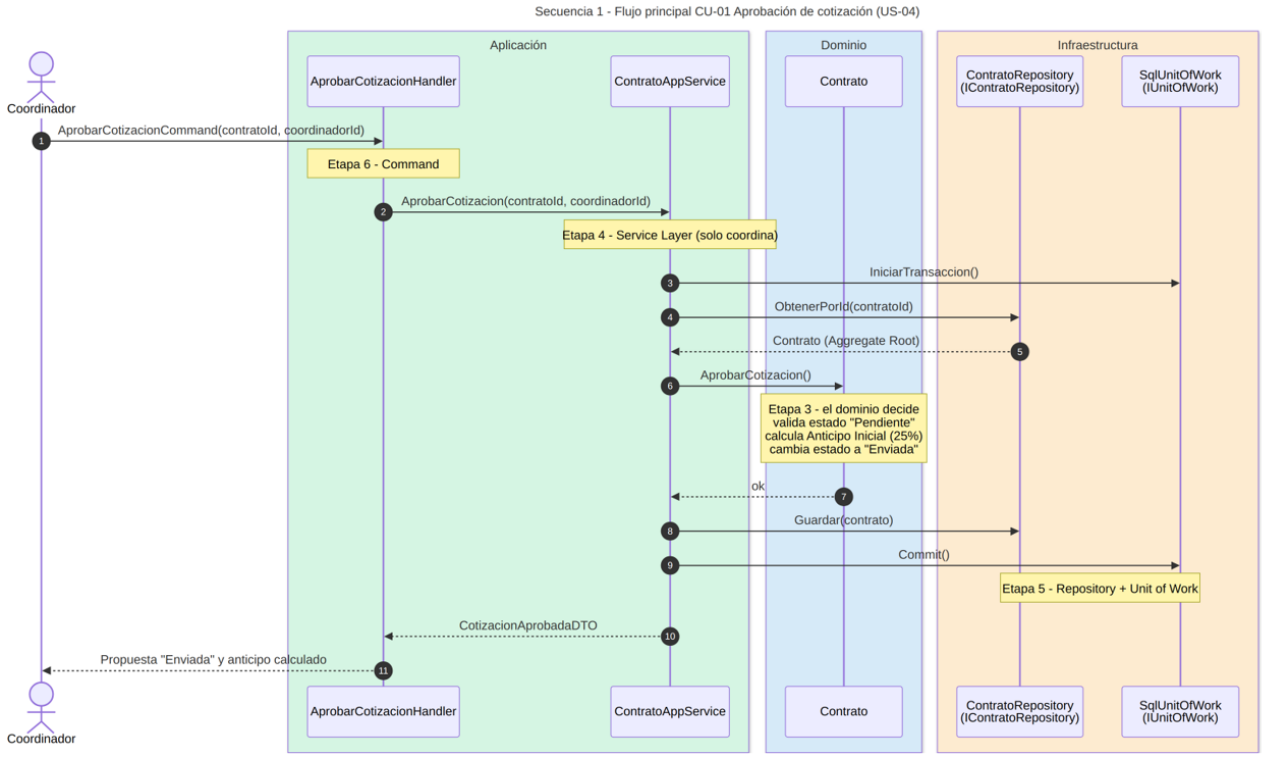


Cisco Confidential




# Diagramas de secuencia

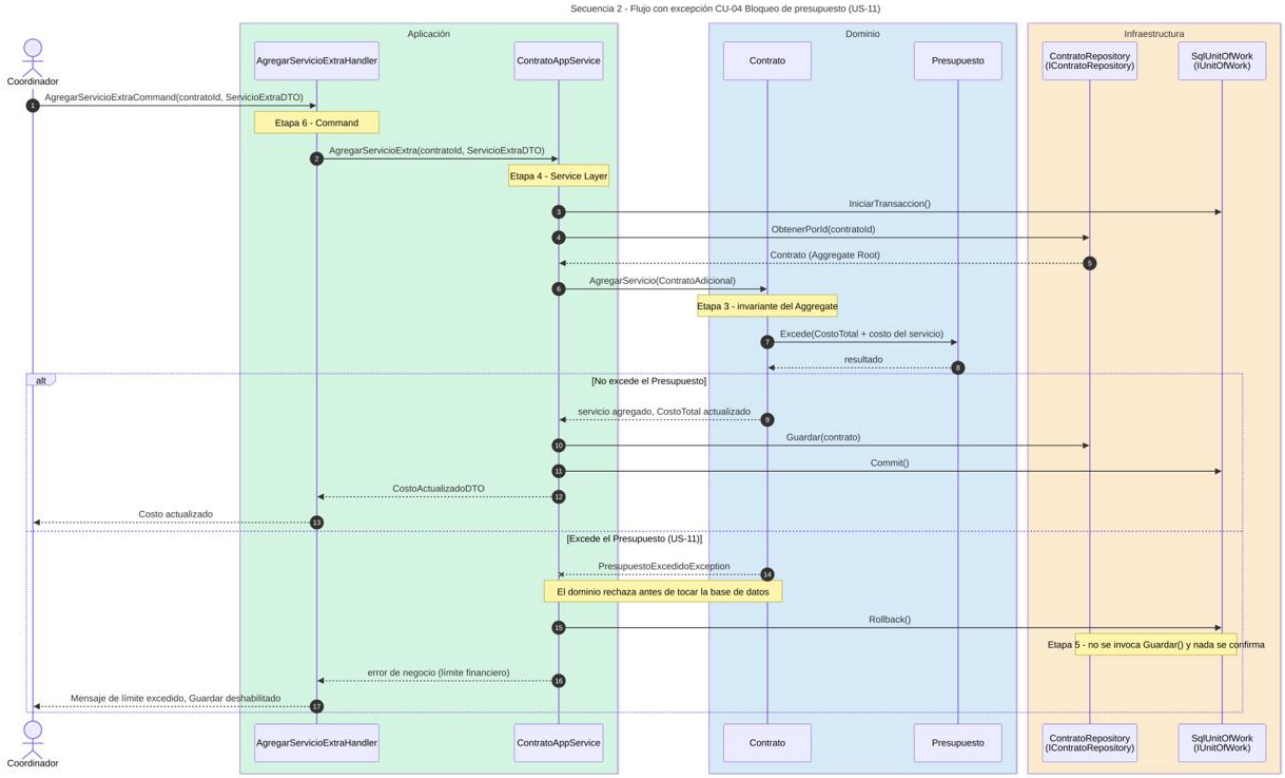
## Secuencia del flujo principal




## Secuencia con excepción



Cisco Confidential




## Architecture Decision Records

Elaboren al menos tres ADR. Cada ADR documentará una decisión que tenga consecuencias para el diseño. Como mínimo, registren la selección del patrón de lógica de negocio y dos decisiones adicionales tomadas durante la actividad.

### ADR-1. Domain Model como patrón de lógica de negocio

| Campo del ADR | Contenido |
|---|---|
| **Título** | Usar Domain Model como patrón principal de la lógica de negocio. |
| **Estado** | Aceptada. |
| **Contexto** | Eventos Nova depende de reglas financieras y temporales que interactúan entre sí: bloqueo de servicios adicionales que excedan el presupuesto (US-11), límite de 10 días para cambios normales (US-08), tolerancia de 2 días y cancelación en cascada por saldo pendiente (US-13), confirmación de proveedores en 3 días y lista de advertencia (US-07). Evidencia de U2: BRD (restricciones y necesidades del negocio) y Glosario (Contrato Activo, Límite de Cambios Normales). |
| **Alternativas** | Transaction Script: simple al inicio, pero dispersa los cálculos de fechas y penalizaciones en funciones sueltas y duplica reglas.<br>Table Module: sirve para CRUD, pero la validación se complica cuando el límite depende del estado de otro registro (fecha del Contrato frente a fecha del Evento).<br>Domain Model: concentra reglas e invariantes en objetos, a cambio de más código y mapeo. |
| **Decisión** | Domain Model. Las reglas viven en Contrato, Evento, Proveedor y PenalizacionDomainService, no en servicios ni en la base de datos. |



Cisco Confidential

| Campo del ADR | Contenido |
|---|---|
| **Consecuencias** | Beneficios: invariantes protegidos en un solo lugar y reglas que pueden cambiar sin tocar la persistencia.<br>Costos: curva de aprendizaje, mapeo objeto-relacional (ver ADR-004) y lecturas masivas costosas, mitigadas con ADR-005.<br>Trabajo posterior: pruebas unitarias del dominio. |
| **Trazabilidad** | US-07, US-08, US-11, US-13; CU-02 a CU-05; BRD (restricciones); Glosario. |

### ADR-2. Contrato como Aggregate Root que incluye Contrato Adicional y Presupuesto

| Campo del ADR | Contenido |
|---|---|
| **Título** | Agrupar Contrato Principal, Contratos Adicionales y Presupuesto bajo la raíz Contrato. |
| **Estado** | Aceptada. |
| **Contexto** | CU-04 exige comparar el costo total (contrato más adicionales) con el presupuesto autorizado, y el cliente puede pedir adicionales días después. Un servicio extra no tiene ciclo de vida sin un evento principal. |
| **Alternativas** | ContratoAdicional como Aggregate independiente: ninguna raíz podría validar el total y se rompería el invariante de US-11.<br>Un único Aggregate con Evento y todos los proveedores: demasiado grande y costoso de cargar. |
| **Decisión** | Contrato es la raíz de un Aggregate con ContratoPrincipal, ContratoAdicional y Presupuesto. Evento y Proveedor quedan fuera y se referencian por Id. |
| **Consecuencias** | Beneficios: el invariante del presupuesto se verifica dentro de una sola transacción.<br>Costos: toda modificación pasa por la raíz y el Aggregate crece con cada adicional.<br>Trabajo posterior: decidir cómo manejar cambios concurrentes sobre el mismo contrato. |
| **Trazabilidad** | US-05, US-10, US-11; CU-04; Glosario (Contrato Principal, Contrato Adicional, Presupuesto Operativo). |

### ADR-3. Service Layer que solo coordina errores de dominio en lenguaje del negocio

| Campo del ADR | Contenido |
|---|---|
| **Título** | Application Services sin reglas de negocio y excepciones de dominio expresadas en términos del negocio. |
| **Estado** | Aceptada. |
| **Contexto** | Cada caso de uso debe recibir la solicitud, cargar objetos, invocar al dominio, persistir y responder, sin que la Service Layer acumule reglas (Etapa 4). Los rechazos deben poder mostrarse al Coordinador o al cliente. |
| **Alternativas** | Reglas dentro de los servicios: se vuelve un Transaction Script disfrazado. |



Cisco Confidential

| | |
|---|---|
| | Reglas en la interfaz: quedan expuestas y duplicadas.<br>Servicios que coordinan y dominio que decide. |
| **Decisión** | Cuatro Application Services con métodos nombrados con intención de negocio (AprobarCotizacion, RegistrarCancelacion, SolicitarCambioNormal, AgregarServicioExtra, AplicarPenalizacion). El dominio lanza PresupuestoExcedidoException o CambioFueraDeTiempoException; el servicio hace Rollback y devuelve un error de negocio. |
| **Consecuencias** | Beneficios: reglas verificables sin infraestructura y nombres alineados con el glosario.<br>Costos: más clases y conversión a DTO.<br>Riesgo: que los servicios crezcan; se revisa en la Etapa 7 (SOLID). |
| **Trazabilidad** | CU-01 a CU-05; US-04, US-07, US-08, US-11, US-13. |

### ADR-4. Repository por Aggregate y Unit of Work en la capa de aplicación

| Campo del ADR | Contenido |
|---|---|
| **Título** | Persistencia detrás de interfaces de Repository y transacción controlada por Unit of Work. |
| **Estado** | Aceptada. |
| **Contexto** | El dominio no debe depender de SQL ni del ORM, y las operaciones de cada caso de uso deben completarse juntas o no aplicarse (Etapa 5). |
| **Alternativas** | Acceso directo a la base de datos desde los servicios: acopla la aplicación a SQL.<br>Repository genérico de CRUD: expone operaciones que el dominio no necesita.<br>Repository por Aggregate con interfaces del dominio y un Unit of Work. |
| **Decisión** | IContratoRepository, IProveedorRepository e IEventoRepository (ObtenerPorId y Guardar) se definen junto al dominio y se implementan en infraestructura. IUnitOfWork (IniciarTransaccion, Commit, Rollback) lo invoca el Application Service. Ante una excepción de dominio se ejecuta Rollback y no se llama a Guardar. |
| **Consecuencias** | Beneficios: dominio sin SQL, base de datos intercambiable y límite transaccional claro.<br>Costos: código de mapeo y sobrecarga del ORM.<br>Trabajo posterior: definir si los Domain Events se despachan antes o después del Commit. |
| **Trazabilidad** | CU-01 a CU-05; hallazgo sobre IUnitOfWork en ProveedorAppService y AdminAppService. |

### ADR-5. CQRS básico sobre una sola base de datos

| Campo del ADR | Contenido |
|---|---|
| **Título** | Separar Commands y Queries sin bases de datos distintas. |
| **Estado** | Aceptada. |
| **Contexto** | Cargar un Aggregate completo solo para leer datos es ineficiente, riesgo señalado en la Etapa 2. Las escrituras sí requieren pasar por el Domain Model. |
| **Alternativas** | Un solo modelo para lectura y escritura: las consultas cargan Aggregates completos.<br>CQRS completo con bases separadas: complejidad desproporcionada para el caso. |



Cisco Confidential

| | |
|---|---|
| | CQRS básico en la misma base de datos. |
| **Decisión** | AprobarCotizacionCommand y AgregarServicioExtraCommand pasan por su Handler hacia el Application Service y el dominio. ConsultarContratoQuery y ConsultarProveedoresQuery usan Handlers que leen mediante IContratoConsultas e IProveedorConsultas y devuelven DTOs de lectura, sin cargar Aggregates. |
| **Consecuencias** | Beneficios: lecturas ligeras y modelos de lectura independientes del de escritura.<br>Costos: más piezas que mantener.<br>Pendiente: justificar que CU-02, CU-03 y CU-05 sigan invocando directamente su Application Service. |
| **Trazabilidad** | CU-01 y CU-04; Etapa 2 (riesgo de lecturas masivas); Etapa 6. |

### Herramientas permitidas

Pueden utilizar draw.io, PlantUML, Mermaid u otra herramienta que permita entregar diagramas legibles. Los archivos fuente deben incluirse en el repositorio; no entreguen únicamente capturas de pantalla.

## 14 Vibe Coding con IA

Usarán IA para convertir descripciones del negocio en alternativas de arquitectura, generar estructura inicial, revisar código y producir borradores de diagramas. El equipo sigue siendo responsable de la arquitectura y debe verificar cada resultado contra los documentos de la Unidad 2.

### Ciclo de trabajo esperado

| Paso | Accion | Evidencia |
|---|---|---|
| 1 | Preparar contexto | Seleccionar fragmentos relevantes del BRD, historias, reglas, BPMN y glosario. |
| 2 | Solicitar propuesta | Pedir una alternativa concreta y exigir que la IA declare supuestos. |
| 3 | Comparar | Contrastar la respuesta con el caso y con los conceptos de la unidad. |
| 4 | Corregir | Modificar nombres, responsabilidades, limites y dependencias que no correspondan. |
| 5 | Validar | Revisar trazabilidad, consistencia entre diagramas y comportamiento esperado. |
| 6 | Registrar | Guardar prompt, respuesta, decision humana y version resultante. |

### Prompts de apoyo

Los siguientes formatos son puntos de partida. Completen los corchetes con información de su empresa y eviten enviar datos que no formen parte del caso académico.



Cisco Confidential

- Patrones: Con base en [reglas, cambios y casos de uso], compara Transaction Script, Table Module y Domain Model. Declara tus supuestos y no elijas una opción hasta mostrar sus consecuencias.
- Dominio: Revisa este glosario y estas reglas [contenido]. Propón candidatos a entidades, Value Objects, Aggregates y Domain Events. Para cada candidato, indica la evidencia utilizada y las dudas que requieren validación humana.
- Service Layer: A partir de estos casos de uso [contenido], propone firmas de Application Services. Separa coordinación de reglas de dominio y explica cualquier responsabilidad dudosa.
- SOLID: Analiza este diseño [contenido] y señala posibles violaciones SOLID. No reescribas todo; identifica evidencia, riesgo y alternativas para que el equipo decida.
- Diagramas: Convierte esta arquitectura validada [contenido] en un borrador PlantUML o Mermaid. No agregues clases, reglas o relaciones que no aparezcan en la descripción.

## Bitacora obligatoria de IA

| Iteracion | Objetivo | Prompt o enlace | Respuesta util | Error o supuesto | Decision del equipo |
|---|---|---|---|---|---|
| 1 | Seleccionar patrón arquitectónico y delimitar 5 casos de uso clave | "Con base en las reglas del BRD de Eventos Nova, sugiere 5 Casos de Uso con flujos de excepción y justifica la elección de Domain Model." | Sugerencia de US 04, 07, 08, 11 y 13 con tabla comparativa a favor de Domain Model. | Error mínimo, la IA modifico ciertas palabras usadas en las users-stories | Se aceptaron las sugerencias de la IA, pero, manualmente se escribieron las user stories de acuerdo con el documento de la unidad 2 |
| 2 | Construir el Mapa de Dominio identificando elementos clave. | "Revisa el Glosario y BRD de Eventos Nova. Propón Entidades, Value Objects, Aggregates (con sus invariantes) y Domain Events para la Etapa 3." | Se mapeó el "Contrato" como Aggregate Root para proteger las reglas de presupuesto y límites de días. | Supuesto: Se asumió que los "Contratos Adicionales" (US 10) viven dentro del Aggregate Root principal para validar el límite financiero global. | Aceptar la agrupación en el Aggregate Contrato para garantizar que la suma total no rompa el invariante del presupuesto autorizado. |
| 3 | Diseñar la Service Layer sin acumular reglas de negocio. | "Diseña los Application Services para los 5 Casos de Uso. El servicio solo debe coordinar. Justifica qué reglas delega al dominio y responde las preguntas de revisión obligatoria." | Se definieron firmas de métodos basados en intenciones (ej. AgregarServicioExtra) e inyección de dependencias mediante interfaces (Repositorios). | Supuesto: Se incluyó un IDomainEventDispatcher en la cancelación de proveedores para orquestar la lista de advertencia de forma asíncrona. | Se acepta el uso del Dispatcher para los Domain Events, manteniendo la Service Layer limpia de lógica procedimental |



Cisco Confidential

| Iteracion | Objetivo | Prompt o enlace | Respuesta util | Error o supuesto | Decision del equipo |
|---|---|---|---|---|---|
| 4 | Generar Diagrama UML de Clases y revisión de principios SOLID. | "Analiza el diseño, señala hallazgos SOLID y genera el código PlantUML del diagrama de clases integrando Dominio, Aplicación e Infraestructura." | Se generó la tabla de los 5 hallazgos SOLID y el script de PlantUML con las flechas de dependencia invertida. | Supuesto: La IA asumió que los Handlers llaman a los AppServices directamente en vez del Dominio. | Aceptamos el modelo. Mantiene la Service Layer como punto central de orquestación transaccional y facilita la lectura del diagrama. |
| 5 | Evaluar la arquitectura completa bajo los cinco principios SOLID para detectar acoplamientos indebidos y corregir responsabilidades. | "Analiza el diseño actual de los repositorios y servicios de Eventos Nova y evalúa posibles violaciones a los principios SOLID, especialmente el principio de Responsabilidad Única (SRP) y Segregación de Interfaces (ISP)." | La IA identificó que una interfaz de repositorio monolítica obligaría a los servicios a depender de métodos que no usan, sugiriendo segregar los contratos por Aggregate Root y separar las consultas de lectura (IContratoConsultas) de los repositorios principales. | Supuesto: La IA sugirió modificar directamente las clases concretas de infraestructura para añadir lógica de caché. | Se rechazó modificar la infraestructura con lógicas ajenas y se aplicó estrictamente el principio de Inversión de Dependencias (DIP), asegurando que la capa de aplicación dependa exclusivamente de abstracciones e interfaces inyectadas. |



Cisco Confidential

| Iteracion | Objetivo | Prompt o enlace | Respuesta util | Error o supuesto | Decision del equipo |
|---|---|---|---|---|---|
| 6 | Organizar y estructurar el repositorio de GitHub separando los entregables formales en directorios limpios (docs/, diagrams) para garantizar la mantenibilidad y correcta revisión del proyecto. | "Propón una estructura de directorios y un archivo README profesional para un repositorio de GitHub que albergue un proyecto bajo arquitectura limpia (Domain Model, Application, Infrastructure) para la empresa Eventos Nova." | La IA sugirió una jerarquía modular basada en carpetas estándar (docs para informes/bitácoras, diagrams para esquemas UML/PlantUML), además de un archivo README.md con instrucciones claras de despliegue y revisión. | Supuesto: La IA intentó generar scripts de automatización de despliegue (CI/CD con GitHub Actions) y archivos de configuración de Docker complejos que excedían el alcance académico de la actividad. | Se rechazó la inclusión de infraestructura de despliegue automatizado por innecesaria. Se conservó únicamente la estructura modular estricta de carpetas y el contenido documental requerido por la rúbrica institucional. |

# 15 Etapa 9 Validar la propuesta

Antes de entregar, recorran cada caso de uso sobre los diagramas y el diseño. El equipo debe poder explicar qué componente recibe la solicitud, dónde se valida cada regla, qué objetos cambian, cómo se persisten los cambios y qué resultado se devuelve.

| Validacion | Pregunta | Resultado del equipo |
|---|---|---|
| Trazabilidad | ¿Cada componente importante responde a una necesidad, regla o caso de uso? | Los componentes propuestos están relacionados con los casos de uso CU-01 al CU-05 y las historias de usuario US-04, US-07, US-08, US-11 y US-13. |
| Lenguaje | ¿Los mismos terminos significan lo mismo en BRD, UML, codigo y ADR? | Se mantienen términos como Contrato, Evento, Proveedor, Presupuesto y Anticipo Inicial para conservar la consistencia entre los documentos y la arquitectura. |
| Responsabilidad | ¿Las reglas permanecen en el dominio o en el patron elegido con una justificacion? | Las reglas de negocio permanecen en las entidades y servicios de dominio. Los Application Services únicamente coordinan las operaciones. |



Cisco Confidential

| Validacion | Pregunta | Resultado del equipo |
|---|---|---|
| Consistencia | ¿Los diagramas de clase y secuencia representan la misma arquitectura? | La arquitectura propuesta mantiene relaciones entre entidades, repositorios y servicios. Los diagramas de clases y secuencia deberán representar estas mismas relaciones. |
| Persistencia | ¿El dominio evita depender directamente de SQL, ORM o detalles de infraestructura? | Se utilizan interfaces Repository para separar el dominio de la base de datos, evitando dependencias directas de SQL o herramientas externas. |
| Transaccion | ¿El limite de la operacion y su manejo de error estan definidos? | Unit of Work permite confirmar las operaciones cuando se cumplen las reglas del negocio y revertir los cambios si ocurre algún error. |
| IA | ¿Cada aporte de IA fue revisado y su uso esta registrado? | Se utilizó inteligencia artificial como apoyo para proponer y revisar la arquitectura. Las propuestas deben compararse con las reglas del negocio y registrar las correcciones realizadas por el equipo. |

# 16 Entregables

| Entregable | Contenido mínimo |
|---|---|
| 01 Informe de arquitectura | Documento PDF con portada, alcance, comparación de patrones, mapa de dominio, Service Layer, Repository, Unit of Work, CQRS, revisión SOLID, ADR y conclusiones. |
| 02 Diagrama de dominio y clases | Archivo fuente y exportación legible en PDF o PNG. |
| 03 Diagramas de secuencia | Dos archivos fuente y sus exportaciones. |
| 04 Matriz de trazabilidad | Relación entre necesidades, reglas, historias, casos de uso, componentes y decisiones. |
| 05 Bitacora de IA | Prompts, respuestas relevantes, errores detectados, correcciones y decisiones humanas. |
| 06 Codigo base | Estructura de carpetas y código mínimo que represente la arquitectura propuesta. |
| 07 Repositorio GitHub | Todos los archivos organizados, historial de commits y README con instrucciones de revisión. |
| 08 Presentacion y defensa | Exposición breve donde todos los integrantes puedan explicar y defender decisiones. |



Cisco Confidential

### Estructura sugerida del repositorio

- docs Informe, ADR, trazabilidad y bitacora de IA
- diagrams Archivos fuente y exportaciones
- src Dominio, aplicacion e infraestructura
- tests Escenarios o pruebas seleccionadas
- README Empresa, alcance, instrucciones y enlaces a entregables

## 17 Rubrica de evaluacion

| Criterio | Evidencia esperada | Puntos |
|---|---|---|
| Continuidad y trazabilidad | Usa los productos de U2, conserva el alcance y relaciona decisiones con evidencia. | 15 |
| Seleccion del patron | Compara alternativas y justifica la eleccion con criterios concretos del caso. | 15 |
| Mapa de dominio | Distingue responsabilidades, limites, lenguaje, invariantes y eventos de forma coherente. | 20 |
| Service Layer y persistencia | Coordina casos de uso, separa reglas y define Repository y Unit of Work. | 15 |
| CQRS y SOLID | Separa operaciones con criterio y revisa el diseño mediante evidencia concreta. | 10 |
| UML y ADR | Los diagramas son consistentes y las decisiones registran contexto, alternativas y consecuencias. | 15 |
| Uso responsable de IA | Presenta iteraciones, detecta errores y explica las decisiones humanas. | 5 |
| Repositorio y defensa | Entrega organizada, legible y defendida por todos los integrantes. | 5 |

**Total 100 puntos**

## 18 Criterios de integridad academica

- La IA puede proponer, revisar y explicar, pero no reemplaza la justificación del equipo.
- Toda afirmación sobre el negocio debe provenir del caso o identificarse como supuesto pendiente de validación.
- El equipo debe conservar la autoría de sus decisiones y poder explicarlas sin leer una respuesta generada.
- El código, los diagramas o los textos generados deben revisarse antes de integrarlos al repositorio.
- Copiar la arquitectura de otro equipo o entregar contenido que no corresponde a la empresa asignada se considera incumplimiento.



Cisco Confidential

# 19 Lista final de comprobación

- [ ] Trabajamos con la misma empresa y el mismo alcance de la Unidad 2.
- [ ] La selección del patrón está respaldada por evidencia del caso.
- [ ] El mapa de dominio usa el lenguaje ubicuo del proyecto.
- [ ] Cada Application Service representa un caso de uso y coordina sin concentrar todas las reglas.
- [ ] Las interfaces de Repository hablan en términos del dominio.
- [ ] Los Commands y Queries tienen responsabilidades distintas y justificadas.
- [ ] Revisamos el diseño con SOLID y corregimos los hallazgos relevantes.
- [ ] Los diagramas de clase y secuencia son consistentes.
- [ ] Incluimos al menos tres ADR y una matriz de trazabilidad actualizada.
- [ ] La bitácora demuestra cómo usamos y validamos la IA.
- [ ] El repositorio contiene archivos fuente, exportaciones y un README claro.
- [ ] Todos los integrantes pueden defender las decisiones tomadas.

# 20 Preguntas para la defensa

- ¿Qué evidencia del negocio influyó más en la selección del patrón?
- ¿Qué responsabilidad fue la más difícil de ubicar y cómo tomaron la decisión?
- ¿Qué regla protege el diseño y qué ocurriría si estuviera en una capa incorrecta?
- ¿Qué cambió después de la revisión con SOLID?
- ¿Qué propuesta de la IA rechazaron o corrigieron y por qué?
- ¿Qué consecuencia negativa aceptaron al tomar su decisión arquitectural principal?



Cisco Confidential